



From Bronze to Business Value Faster with Materialized Lakehouse Views

Understanding materialization, refresh behavior, scheduling, and observability



LinkedIn

 /mikediehlsqbi

Mike Diehl

Director, Data Engineering & Business Intelligence

Mike.Diehl@Improving.Com

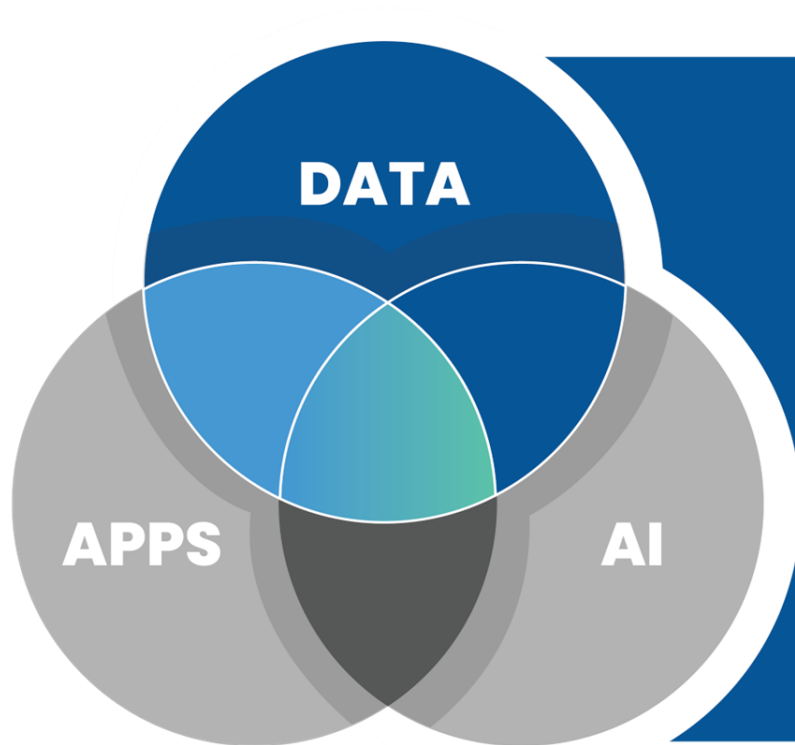


Session materials

<https://github.com/xhead/Fabric-mlv-demo-2026>

improving

It's what we do.™



Data

Build a modern data foundation that scales with your business. From cloud data platforms to advanced analytics, we unlock actionable insights through secure, well-architected solutions.

- Real-Time Data Systems
- Data Platform Modernization
- BI & Analytics
- Integration & Engineering

CANADA
VANCOUVER
CALGARY
WINNIPEG
OTTAWA
TORONTO

MEXICO
GUADALAJARA
AGUASCALIENTES

CENTRAL AMERICA
GUATEMALA

SOUTH AMERICA
SANTIAGO
BUENOS AIRES

INDIA
PUNE

UNITED STATES
MINNEAPOLIS
OMAHA
CHICAGO (2)
COLUMBUS
CLEVELAND
ATLANTA
DALLAS
AUSTIN
HOUSTON

The medallion tax

WHAT IS

Build it once. Pay for it forever.

Notebook sprawl

- One notebook per table
- Patterns drift

Pipeline plumbing

- Orchestration per table
- Watermarks and retries

Repeated query work

- Joins re-run per consumer
- Aggregations recomputed

Refresh uncertainty

- Unclear compute cost
- Unclear freshness

The tax is not one big cost — it is a small recurring cost multiplied by every table and every consumer.

What is a materialized view?

DEFINITION

Regular SQL view

A stored query definition. No data of its own.

```
CREATE VIEW gold.OrderDetail AS  
SELECT ... FROM silver.Orders o  
JOIN silver.Products p ON ...
```

- Runs at query time
- Every consumer pays the full cost
- Always current

Materialized lakehouse view

A query definition plus a persisted Delta result.

```
CREATE MATERIALIZED LAKE VIEW  
  gold.OrderDetail AS  
SELECT ... FROM silver.Orders o ...
```

- Runs at refresh time
- Consumers read stored results
- Fresh as of the last refresh
- Usable by DirectLake semantic models

One word changes the economics: a view computes on read; a materialized view computes on refresh and stores the answer.

Where MLVs fit in the architecture

ORIENTATION

Bronze

Often stays procedural

- External APIs and files
- Watermarks and state

Notebooks / pipelines

Silver

Some MLV candidates

- Conforming and cleansing
- Declarative joins

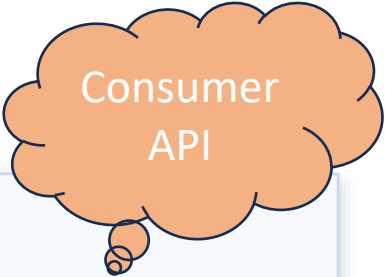
Mixed

Gold

Primary MLV use case

- Business aggregates
- Repeatedly queried results

MLV-first



Consumer
API

The production-inspired case study in this session uses MLVs primarily for the Gold layer.

Why materialize a complex query?

WHAT COULD BE

Pay once, not per consumer

Expensive work happens at refresh, not in every query.

Many readers, one result

Every consumer reads the same persisted table.

A stable Gold contract

A predictable, governed shape for downstream users.

**Materialization shifts compute.
It does not eliminate compute.**

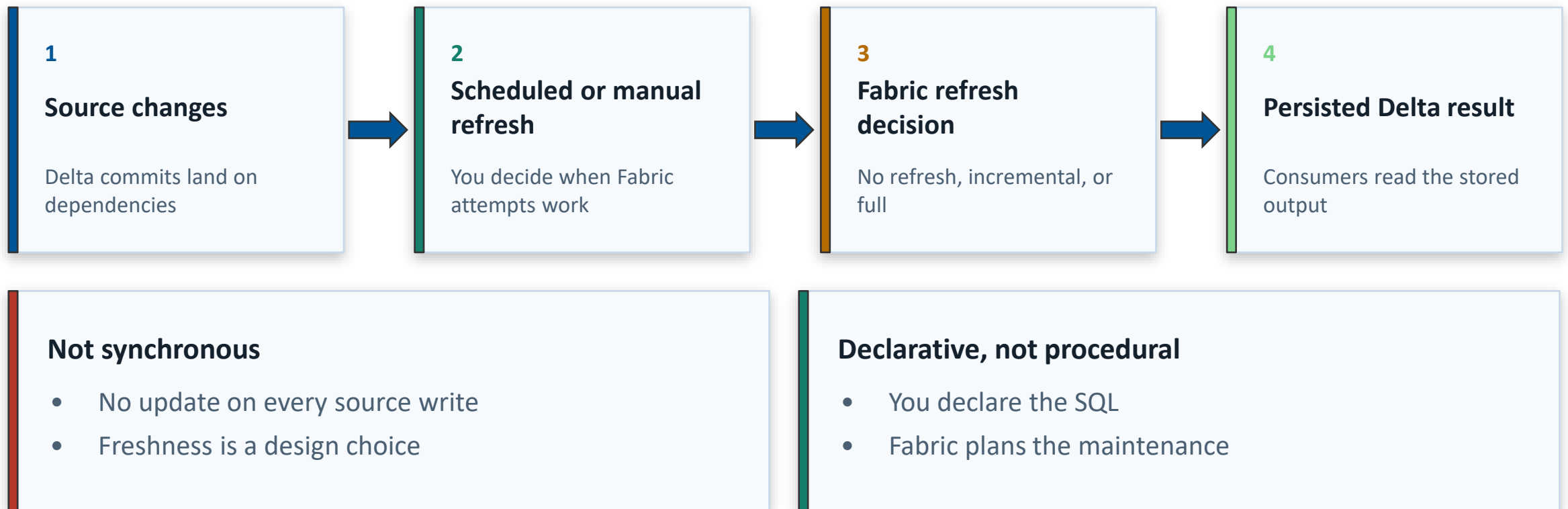
- Query cost goes down
- Refresh cost appears on a schedule
- The win comes from reuse

If a result is queried once a week and refreshed hourly, materialization is the wrong trade. Reuse is the deciding factor. But also, "it depends".

What makes a Fabric MLV different?

REFRESH MODEL

Not continuously maintained. Maintained by a refresh — scheduled or triggered.



The MLV maintenance tradeoff

TENSION

The key question: how much work is required to keep the persisted result current?

What you gain

Faster consumer queries

Already computed. Already governed. Already shaped for the business.

What you now own

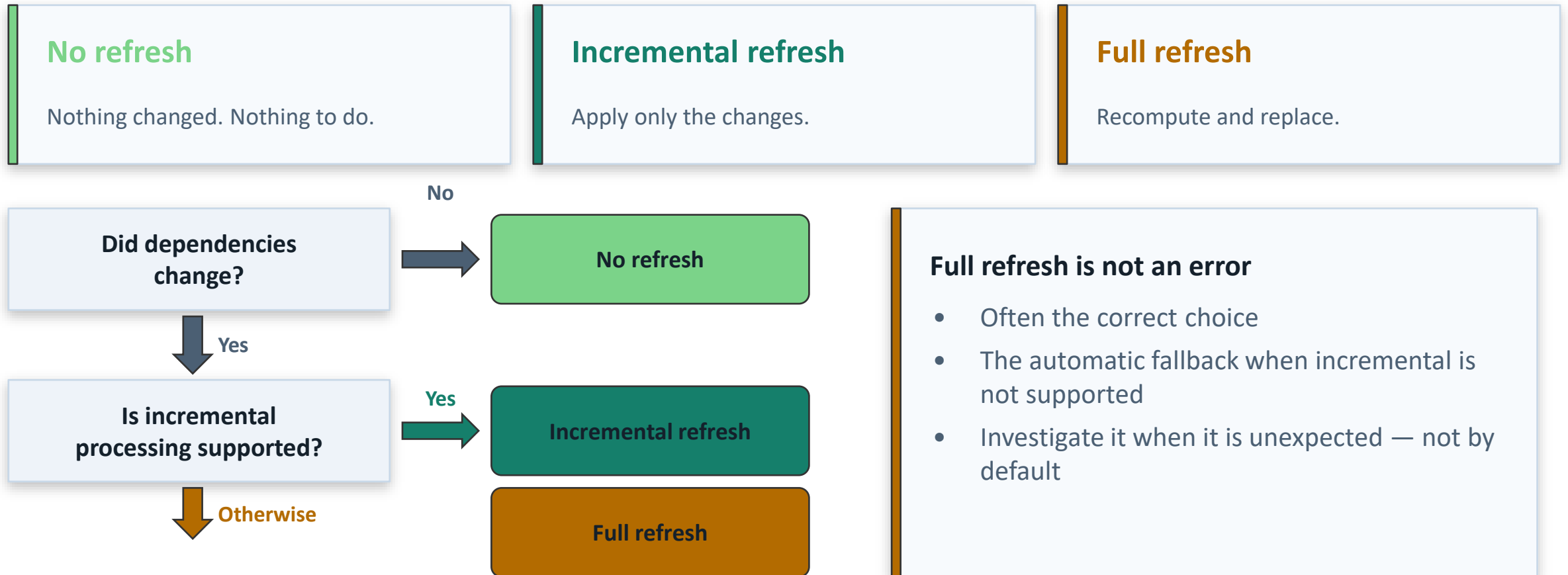
Refresh compute	Work on every attempted refresh
Storage	A second copy of the result
Freshness	Bounded by cadence
Operational monitoring	Someone watches the runs

Materialization is not free — it is a trade you make consciously, per data product.

Fabric optimal refresh

RESOLUTION

Fabric picks the least expensive maintenance it can support — per run.



Change Data Feed and append-only changes

MECHANICS

Delta commits

- Every source write leaves a commit
- Commits say: something changed

Change Data Feed

- Row-level change detail
- Must be enabled on dependencies

Change shape matters

- Append-only is the best case
- Updates and deletes are harder

Append-only source changes

New rows can be folded in without reevaluating what already exists.

Updates and deletes

Changed or removed rows can invalidate what you already persisted.

CDF enables incremental refresh. It does not guarantee incremental refresh.

SQL example: an incremental-refresh candidate

PATTERN

```
SELECT
    o.OrderId,
    o.OrderDate,
    o.CustomerId,
    o.ProductId,
    o.Quantity,
    o.Quantity * p.UnitPrice AS ExtendedAmount
FROM silver.Orders o
INNER JOIN silver.Products p
    ON o.ProductId = p.ProductId;
```

Assumptions behind this being incremental-friendly

- Delta source tables
- CDF enabled on dependencies
- Append-only refresh cycle
- Deterministic expressions only
- Supported query constructs

Why it fits: a new order row produces a new output row. Rows already persisted are unaffected, so the engine can process just the change.

Even with all five assumptions met, the refresh mode is Fabric's decision per run — verify it in Recent runs rather than assuming.

SQL features that can cause a full refresh

CONSTRAINTS

The MLV still gets created. The cost shows up later, as a refresh mode.

Window functions	ROW_NUMBER, RANK, LAG, LEAD	Right-side changes in a LEFT JOIN	Can flip previously unmatched rows
Non-deterministic functions	RAND, UUID, CURRENT_TIMESTAMP	Aggregates needing partitioning conditions	AVG, STDDEV and similar
DISTINCT	Deduplication spans existing rows	PySpark-defined MLVs	Outside the declarative SQL path
Scalar and EXISTS subqueries	Per-row lookups into other data	Non-Delta source tables	No commits, no change feed

One reason underneath all of them: a new row can change the answer for rows you already persisted.

Creation still succeeds — the cost shows up later as a refresh mode, which is why observability matters.

A full-refresh example

WALKTHROUGH

```
WITH RankedOrders AS (  
  SELECT  
    o.*,  
    ROW_NUMBER() OVER (  
      PARTITION BY CustomerId  
      ORDER BY OrderDate DESC  
    ) AS OrderRank  
  FROM silver.Orders o  
)  
SELECT *  
FROM RankedOrders  
WHERE OrderRank = 1;
```

One new order rewrites history

- The MLV holds each customer's latest order
- A newer order arrives
- The old rank-1 row must be removed, not added to

The change is not local to new rows

Before refresh: Customer 42 → Order A (rank 1)

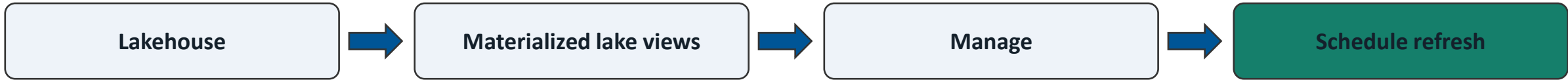
Order B arrives



After refresh: Customer 42 → Order B (rank 1); Order A removed

Start with the simple path: Fabric portal scheduling

SCHEDULING



What a schedule actually says

- Cadence: how often to attempt
- Time zone: when attempts land
- Enabled: whether they happen at all

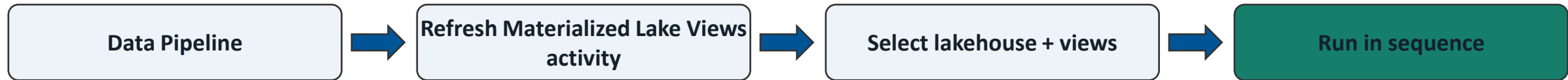
Schedules can also be event-triggered

- OneLake file change events can start a run
- Refresh fires when new data lands, not on a clock
- Can specify a path to watch for events

A schedule answers “when should Fabric attempt a refresh?” — it does not guarantee a successful, or an incremental, refresh.

Orchestrate it: the Pipeline activity that refreshes MLVs

ORCHESTRATION



What the activity does

- Triggers refresh of selected MLVs in a lakehouse
- Runs as a step inside a normal Fabric pipeline
- Waits for completion, so downstream steps see fresh data

Why use a pipeline instead

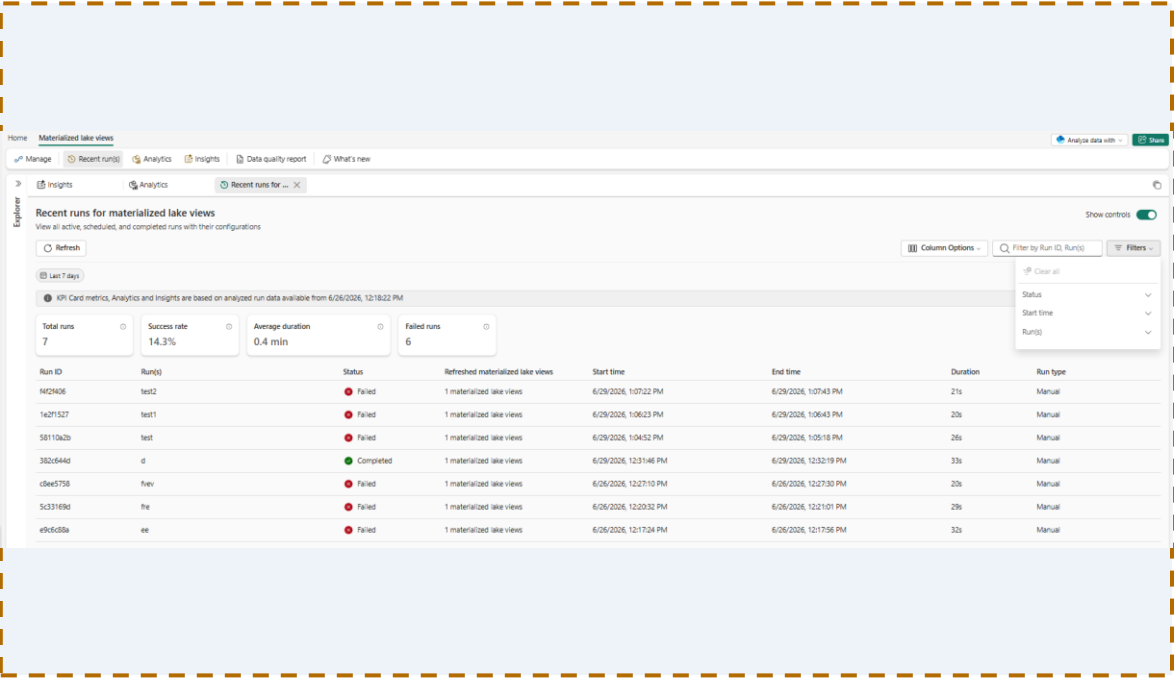
- Chain refresh with ingestion, notebooks, and alerts
- Add retries, parameters, and failure handling
- One orchestration owns the whole end-to-end run

Use the portal schedule for simple cases; use the pipeline activity when refresh must be coordinated with other work.

What a scheduled refresh actually did

OBSERVABILITY

“The schedule tells us what we asked Fabric to do. Recent runs tell us what Fabric actually did.”



Run ID	Run()	Status	Refreshed materialized lake views	Start time	End time	Duration	Run type
1d29406	test2	Failed	1 materialized lake views	6/29/2026, 1:07:22 PM	6/29/2026, 1:07:43 PM	21s	Manual
1e21527	test1	Failed	1 materialized lake views	6/29/2026, 1:06:23 PM	6/29/2026, 1:06:43 PM	20s	Manual
58110a2b	test	Failed	1 materialized lake views	6/29/2026, 1:04:52 PM	6/29/2026, 1:05:18 PM	26s	Manual
382d64d3	d	Completed	1 materialized lake views	6/29/2026, 12:31:46 PM	6/29/2026, 12:32:19 PM	33s	Manual
cd4e5758	frv	Failed	1 materialized lake views	6/26/2026, 12:27:10 PM	6/26/2026, 12:27:30 PM	20s	Manual
5c33169d	frv	Failed	1 materialized lake views	6/26/2026, 12:20:32 PM	6/26/2026, 12:21:01 PM	29s	Manual
e9c6c8a	ee	Failed	1 materialized lake views	6/26/2026, 12:17:04 PM	6/26/2026, 12:17:56 PM	32s	Manual

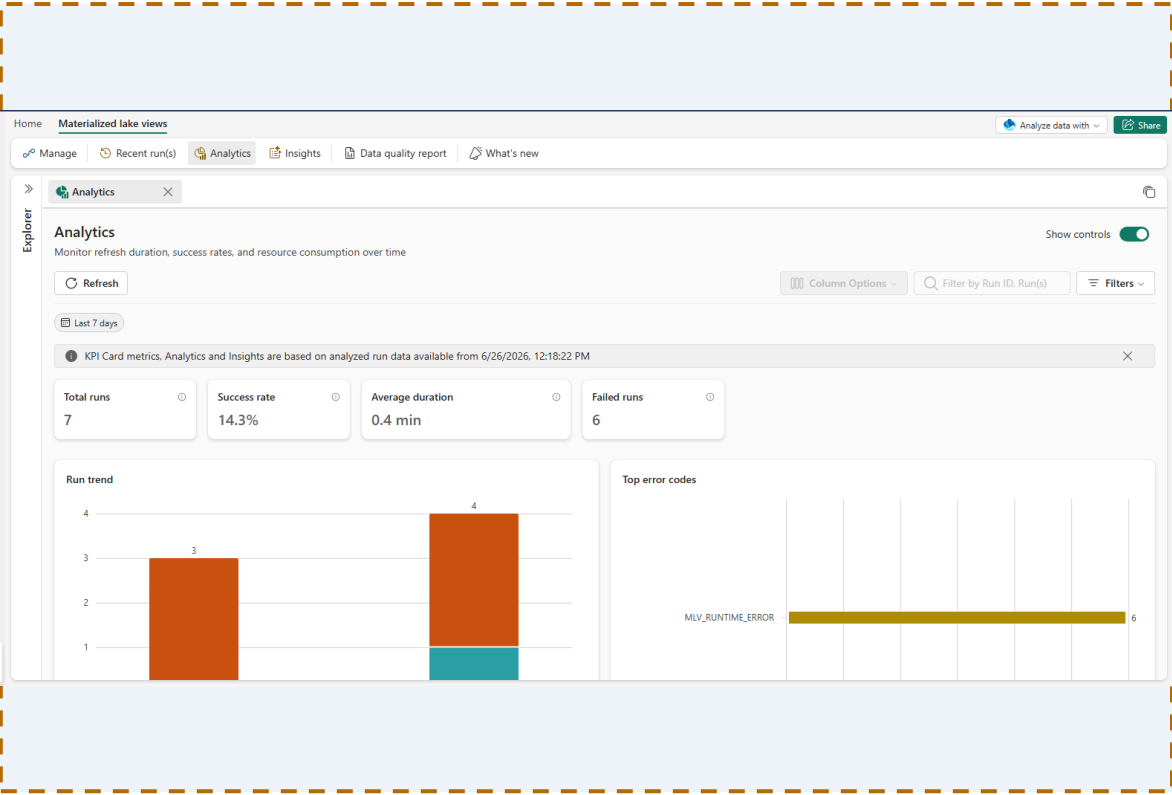
Fabric Recent runs list for a materialized lake view

Run ID	Start / end time
Duration	Status
Trigger type	Affected MLV
Refresh policy or mode	Error information

Values shown during the session will come from the demo environment — no numbers are quoted here in advance.

Refresh analytics and trends

OBSERVABILITY



Fabric Analytics view for materialized lake view refreshes

Run counts	Volume of attempts
Success and failure trends	Reliability direction
Average duration	What “normal” looks like
Full vs incremental vs none	The efficiency signal
Error categories	Where failures cluster
Duration trends	Early warning of cost

No results are invented here — trends will be read live from the demo and production-inspired environments.

Insights and operational diagnosis

OBSERVABILITY

Missing CDF

No row-level change data

Repeated failures

A failing pattern, not an incident

Schema changes

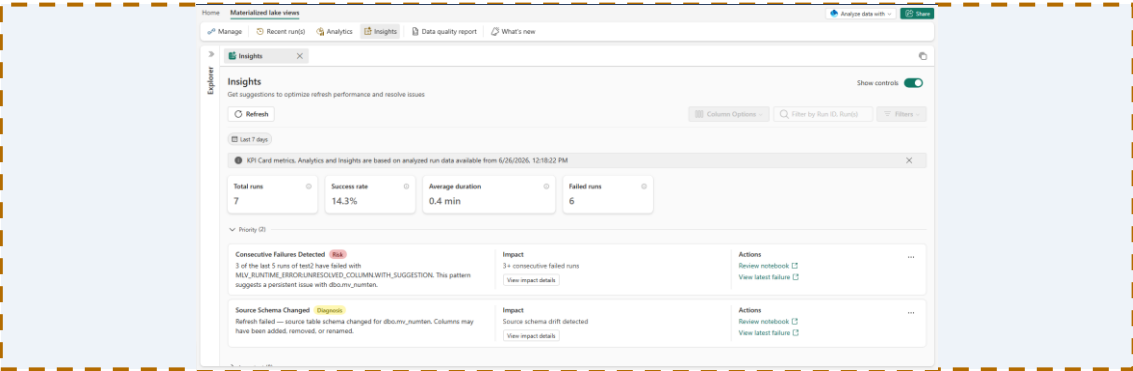
Upstream shape moved

Unsupported conditions

Why incremental was not possible

Optimization opportunities

Where refresh could be cheaper



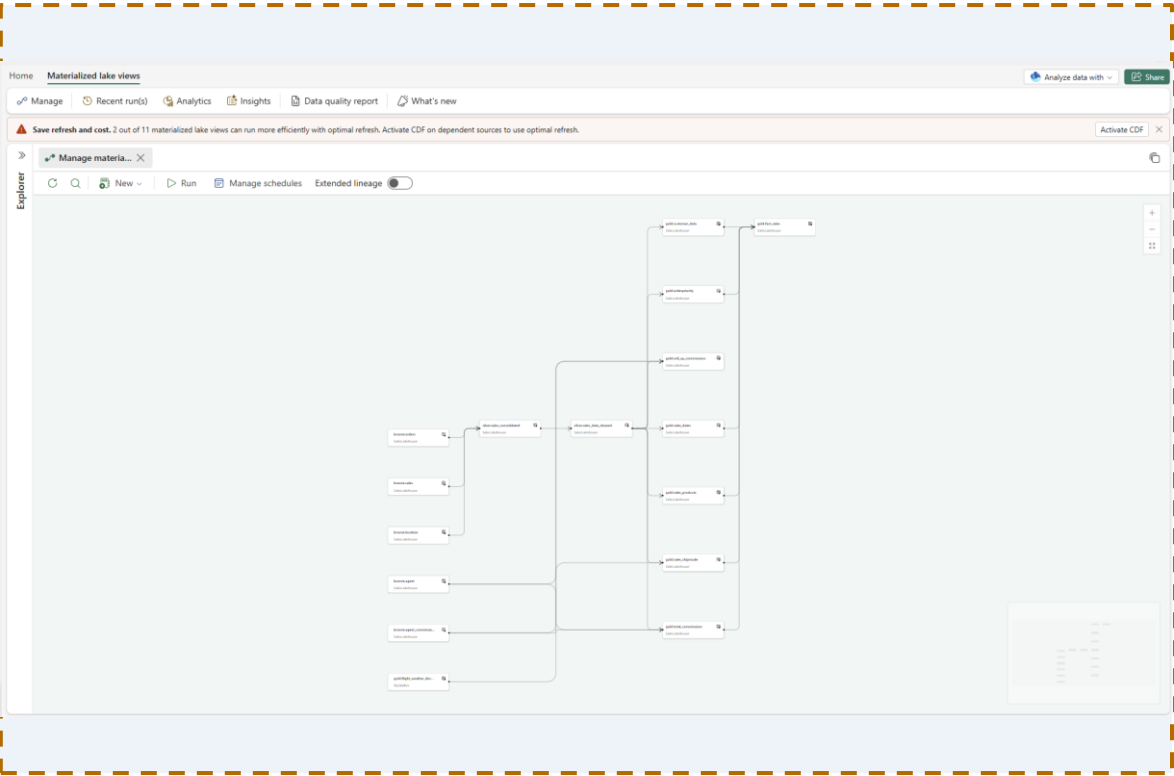
Fabric Insights surface for a materialized lake view

Observability separates two very different statements

- “The schedule ran.”
- “The data product was maintained efficiently.”

Lineage: why a refresh involved these objects

OBSERVABILITY



Fabric lineage view across sources, MLVs, and consumers

Upstream source tables	What feeds the view
MLV dependencies	Views built on views
Shared dependencies	One change, many refreshes
Downstream consumers	Who feels a failed refresh
Recent runs = what happened over time Lineage = why those objects are connected	

Lineage complements refresh history: history explains the timeline, lineage explains the blast radius.

Source-side SQL and Delta evidence

DEEPER INSPECTION

There is no single SQL system table for refresh history — but the source side still talks.

```
SHOW TBLPROPERTIES silver.Sales;
```

Is CDF actually enabled?

```
DESCRIBE HISTORY silver.Sales;
```

What operations hit the source?

```
DESCRIBE DETAIL silver.Sales;
```

Table metadata and size

The screenshot displays the 'Access history' for the user 'alice' in the AWS IAM console. The table lists 18 events, all of which were successful. The events include creating a user, creating a group, and creating an access key. The 'operationMetrics' column shows a 'Success' status for all events.

Id	Timestamp	EventName	Resource	OperationName	Principal	SourceIP	SourceAccount	DestinationAccount	DestinationRegion	OperationStatus	OperationMetrics
1	2018-10-20T17:15:00Z	CreateUser	aws-iam	CreateUser	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
2	2018-10-20T17:15:00Z	CreateGroup	aws-iam	CreateGroup	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
3	2018-10-20T17:15:00Z	CreateUser	aws-iam	CreateUser	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
4	2018-10-20T17:15:00Z	CreateGroup	aws-iam	CreateGroup	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
5	2018-10-20T17:15:00Z	CreateUser	aws-iam	CreateUser	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
6	2018-10-20T17:15:00Z	CreateGroup	aws-iam	CreateGroup	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
7	2018-10-20T17:15:00Z	CreateUser	aws-iam	CreateUser	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
8	2018-10-20T17:15:00Z	CreateGroup	aws-iam	CreateGroup	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
9	2018-10-20T17:15:00Z	CreateUser	aws-iam	CreateUser	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
10	2018-10-20T17:15:00Z	CreateGroup	aws-iam	CreateGroup	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
11	2018-10-20T17:15:00Z	CreateUser	aws-iam	CreateUser	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
12	2018-10-20T17:15:00Z	CreateGroup	aws-iam	CreateGroup	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
13	2018-10-20T17:15:00Z	CreateUser	aws-iam	CreateUser	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
14	2018-10-20T17:15:00Z	CreateGroup	aws-iam	CreateGroup	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
15	2018-10-20T17:15:00Z	CreateUser	aws-iam	CreateUser	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
16	2018-10-20T17:15:00Z	CreateGroup	aws-iam	CreateGroup	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
17	2018-10-20T17:15:00Z	CreateUser	aws-iam	CreateUser	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success
18	2018-10-20T17:15:00Z	CreateGroup	aws-iam	CreateGroup	alice	193.102.250.100	us-east-1	us-east-1	us-east-1	Success	Success

Commit history for a source dependency

Two different questions

- Source Delta history = what happened to the source
- MLV Recent runs = what happened during the refresh

Demo

LIVE DEMO

What we'll walk through in Fabric

- Create a materialized lake view over the lakehouse
- Manage it and set a refresh schedule
- Watch refresh behavior and lineage update
- Show a pipeline MLV Refresh task
- Update some data, look at how it affects the refresh

A practical decision framework

ADOPTION

Use an MLV when

- The transformation is declarative SQL
- Consumers use the result repeatedly
- Sources are reliable Delta tables
- Scheduled refresh meets the freshness need
- The refresh cost is acceptable

Consider a notebook or pipeline when

- Ingestion needs APIs, files, or watermarks
- The logic is procedural, not declarative
- The transformation is stateful or operational
- Full refresh every run, and it hurts

This is not either/or — most mature lakehouses run both, with MLVs concentrated where the SQL is declarative and the result is reused.

Three takeaways

CLOSE

1

Materialization moves compute

From every consumer query to one controlled refresh.

2

CDF enables — it does not guarantee

Your SQL and your source behavior decide whether incremental is possible.

3

Scheduling is only the beginning

Recent runs, Analytics, Insights, Lineage, and Delta history tell you what really happened.



From Bronze to Business Value Faster with Materialized Lakehouse Views

Understanding materialization, refresh behavior, scheduling, and observability



LinkedIn

 /mikediehlsqbi

Mike Diehl

Director, Data Engineering & Business Intelligence

Mike.Diehl@Improving.Com



Session materials

<https://github.com/xhead/Fabric-mlv-demo-2026>